Where Does Agent Reliability Come From?

A Cross-Benchmark Decomposition of Verification Loops, Specialist Models, and Scaffolding in a Production Enterprise Agent

Arunabh Dastidar and the Leni Team
Leni Inc.
Correspondence: arunabh@leni.co

June 2026 (preprint)

Abstract

Multi-step enterprise agent tasks fail in a characteristic way: single-pass inference has no checkpoint between deciding an answer and committing to it. We study one production system (Leni, an AI business analyst) whose architecture installs such checkpoints: *verification loops* (execute, observe, compare, correct) staffed by lightweight task-specialized post-trained models. We evaluate the unmodified production configuration on three public benchmarks stressing distinct failure modes: SpreadsheetBench Verified (silent computation error), BullshitBench v2 (premise confabulation), and the GAIA validation split (cascade error over long tool chains). The full system improves over its frontier base model by +11.0 pp on SpreadsheetBench (91.25% vs. 80.25%, n=400, $p < 0.001$), +7 to +10 pp on BullshitBench (98% vs. 91%, n=100, nominally significant), and ∼+15 pp on GAIA validation (75.2% pass@1 vs. ∼60%, n=165; 83.0% best-of-k). The GAIA figures correct an earlier company report whose 77.6% mixed single-attempt and best-of-k selection across tiers; re-grading every stored trajectory with the official scorer (validated at 100% agreement against the 218 previously labeled runs) yields the numbers here. Our central contribution, however, is a *decomposition* of that uplift: most of it comes from scaffolding, routing, and specialist models rather than from the verification step itself, whose isolated contribution is small (+1.5 pp measured on SpreadsheetBench) but concentrated exactly at the top of the score distribution, where it converts otherwise-failing tasks. We instrument the deterministic loop end-to-end, yielding an empirical verifier confusion matrix (task-level catch rate $c \approx 0.20$, fix rate $r \approx 0.75$, no observed false-alarm regressions) that grounds a compounding-reliability model extended to include verifier false alarms. Preliminary specialist-swap ablations suggest the loop's value depends on *who observes*: replacing the small trained verifier with the generating frontier model eliminates most rescues, consistent with self-assessment biases reported in prior work. A valid-premise control (100 legitimate expert-level questions through the same firewall-active harness) shows zero over-rejections, bounding the firewall's false-positive rate at $\lesssim$3.6% on that distribution. We report all results with confidence intervals, state which cross-system comparisons are and are not statistically resolvable, and enumerate the experiments (an adversarially matched valid-premise control, hidden-test-set submission, external scaffold baselines) required before stronger claims can be made. This is a vendor evaluation of the vendor's own system; the limitations section says so and lists what independent replication would require.

Keywords: LLM agents, reliability, verification, self-correction, orchestration, enterprise AI, benchmarking

1 Introduction

Enterprise deployment of language-model agents fails in a characteristic way. A single tool call works. A two-step chain usually works. By the fifth step, or the first fabricated premise, or the first silently miscomputed cell, the system produces a fluent, confident, and wrong result. In consumer chat this is tolerable; in financial close, contract review, diligence, or reconciliation, a confidently wrong output propagates into real decisions and real liability.

A common response is to wait for a stronger base model. This paper examines a different lever: the architecture wrapped around the model. We study one production system whose design centers on *verification loops*, a control structure that executes an action, independently observes the result, compares the observation against intent, and corrects before committing. Each loop stage is assigned to the lightest model that can perform it reliably, including small (0.5–3B) post-trained specialists.

Prior work has shown both that structured feedback can improve LLM outputs [4, 5, 8, 6] and, pointedly, that models are poor at correcting their own reasoning without external signals [7]. We do not try to settle that debate in general. We measure, inside a single production system under identical conditions, *where* the reliability of a deployed agent comes from, and find three things:

1. **Total architectural uplift is large and, on two of three benchmarks, statistically unambiguous** relative to the bare base model (+11.0 pp on SpreadsheetBench, n=400; +7 to +10 pp on BullshitBench, n=100; ∼+15 pp on GAIA validation, n=165). The comparison class matters: an agentic system (tools, multiple inference passes, multiple models) against raw single-pass inference. This measures the value of the architecture as a whole rather than of any single mechanism (§6).

2. **Decomposition shows the verification step's isolated contribution is small but positionally decisive.** On SpreadsheetBench, scaffolding and prompting account for +9.5 pp of the +11.0 pp uplift; the deterministic recalculation loop adds the final +1.5 pp by rescuing 6 tasks, which is the difference between a mid-leaderboard and a near-top result. On GAIA, after correcting the headline score to a pre-specified pass@1 (§6.3), the loop's marginal contribution over the estimated structure tiers is ∼+1 pp and cannot yet be cleanly isolated (§7).

3. **Who observes matters as much as whether observation happens.** In preliminary specialist-swap runs, moving the observe/compare stage from a small post-trained verifier back onto the frontier model that generated the artifact reduces SpreadsheetBench rescues from 6 tasks to 2, and reduces BullshitBench correct rejection by 4–5 pp. This is consistent with self-preference and self-assessment biases documented for LLM evaluators [12, 11], and suggests that what carries the effect is the independence of the observer, not just the presence of a loop (§7).

We also contribute (i) a formalization of the verification loop with a taxonomy of verification *oracles* (deterministic, self-reflective, planner-mediated) and a compounding-reliability model extended to account for verifier false alarms (§3); (ii) the first end-to-end instrumentation of a production deterministic loop, yielding an empirical verifier confusion matrix (§6.2); and (iii) an explicit accounting of which of our cross-system comparisons are statistically resolvable at current sample sizes, and which are not (§6.3, §10).

What this paper does not claim. We do not claim that verification loops alone explain the measured uplift; our own decomposition shows otherwise. We do not claim statistically significant superiority over other agentic systems on GAIA; the differences are within confidence intervals against

reconstructed competitor figures. On the self-reflective firewall, we report a measured valid-premise control: 100 legitimate, expert-level questions run through the same firewall-active production harness produced zero refusal-like outputs (§6.4); what remains open is a control styled adversarially after the benchmark's own deception techniques. These boundaries are drawn deliberately: the durable finding is the decomposition, not a leaderboard position.

2 Related Work

Self-correction and its limits. Self-Refine [4] and Reflexion [5] show that iterated self-feedback can improve outputs on some tasks; self-consistency [6] improves reliability by sampling and voting. Huang et al. [7] demonstrate that LLMs largely cannot correct their own *reasoning* without external information, and that naive self-critique can degrade performance. Our results are consistent with both lines: our deterministic loop supplies exactly the external signal Huang et al. identify as necessary (a recomputation engine), and our specialist-swap ablation suggests that even with external signals, a generator asked to evaluate its own artifact underperforms an independent observer, echoing self-preference effects in LLM evaluators [12].

Tool-grounded verification. CRITIC [8] interleaves generation with tool-based critique; ReAct [9] interleaves reasoning and acting; AlphaCodium [10] shows test-based iteration dominating single-shot generation for code. We extend this direction in three ways: we isolate the *observation* step (independent read-back through a separate code path) as the load-bearing component; we organize instantiations by their verification oracle; and we measure the pattern in one production system across three unrelated benchmarks rather than per-task-tuned setups.

Neurosymbolic verification. Deterministic engines that re-execute artifacts represent the strongest oracle class. On SpreadsheetBench the current leader is a neurosymbolic system with a custom spreadsheet runtime. We show a commodity deterministic oracle (LibreOffice headless recalculation) recovers most of that benefit at far lower engineering cost, and we quantify the residual gap attributable to LLM-mediated (rather than symbolically enforced) comparison (§9).

LLM-as-judge. BullshitBench is scored by an LLM judge panel. Judge bias, including intra-family preference, is documented [11, 12]; our panel spans three providers, but two of our configurations are Claude-based and one judge is a Claude model. We treat judge-based scores as noisier than exact-match scores throughout and flag this in §10.

Benchmarks. SpreadsheetBench [1] measures exact-match spreadsheet manipulation; we use the expert-annotated Verified subset (400 tasks). GAIA [2] measures long-horizon tool orchestration with exact-match scoring across three difficulty tiers. BullshitBench v2 [3] measures whether a system rejects fabricated premises rather than elaborating on them; we note it is a community benchmark (n=100) and weight conclusions accordingly. The three were chosen because they stress unrelated failure modes (silent computation error, sycophantic confabulation, cascade error), so an architectural pattern that helps all three is evidence of generality.

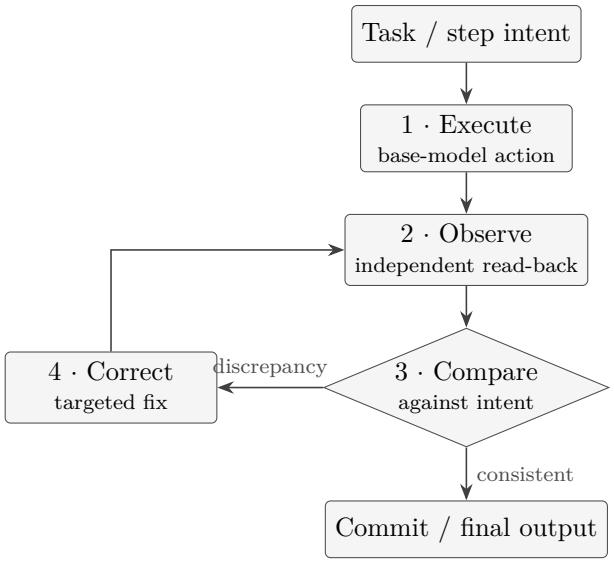

Figure 1: The general verification loop. Observation (stage 2) is what makes defects that are invisible to the generating process visible; correction (stage 4) re-enters observation, not execution.

3 The Verification Loop: Definition and a Reliability Model

3.1 Definition

A *verification loop* is a control structure wrapped around a base-model action with four stages:

1. **Execute**: the base model performs the action (edit a workbook, answer a question, run a tool step).
2. **Observe**: the system produces an independent observation of the result, through a different code path than the one that generated it.
3. **Compare**: the observation is checked against the original intent (task instruction, plan, or premise set).
4. **Correct**: on a detected discrepancy, the system issues a targeted fix and re-observes; on consistency, it commits.

The load-bearing stage is *observation*. Many defects (mutation-order artifacts, serialization corruption, a formula that evaluates differently than written, a premise the model half-noticed was false) are structurally invisible to the process that created them, and become observable only when the output is re-read through an independent path. The rest of the loop exists to force and act on that observation. Figure 1 shows the structure; correction re-enters observation rather than the original execution path.

3.2 A compounding-reliability model with imperfect verification

Consider a task of n dependent steps, each succeeding independently with probability p. Without verification, end-to-end reliability is $R_{\text{base}} = p^n$, which decays geometrically: at $p = 0.95$, a ten-step task succeeds only $\sim$60% of the time.

A per-step verification loop is characterized by four empirical parameters: the *catch rate* c (probability a true error is flagged), the *fix rate* r (probability a flagged true error is repaired), the *false-alarm rate* f (probability a correct step is flagged), and the *breakage rate* b (probability a false-alarm "correction" damages a correct step). Effective per-step reliability becomes

$$p' = \underbrace{p\,(1 - f b)}_{\text{correct steps that survive}} + \underbrace{(1-p)\,c\,r}_{\text{errors caught and fixed}}, \tag{1}$$

and end-to-end reliability $R_{\text{loop}} = (p')^n$. The loop helps if and only if $(1-p)\,c\,r > p\,f\,b$: an imperfect verifier can *reduce* reliability if it frequently second-guesses correct work and its corrections are destructive. Because a beneficial correction acts before the error propagates, the benefit compounds multiplicatively, so the loop's marginal value grows with chain length n.

Two consequences follow. The model predicts that loops matter most on long dependency chains; our data does not yet test this prediction cleanly (§6.3, §7), and we state it as a falsifiable hypothesis rather than a finding. It also makes the verifier's error profile a first-class quantity, and §6.2 reports what appears to be the first published empirical estimate of (c, r, f) from a production deterministic loop.

3.3 The oracle determines the ceiling

The compare stage requires a source of truth: the *verification oracle*. It is the most consequential design choice because it bounds c (what can be caught) and f (what is spuriously flagged). We distinguish three classes (Table 1).

Oracle class	**Source of truth**	**Ceiling**	**Cost**	**Instantiation**
Deterministic	External engine re-executes the artifact	Highest (near-symbolic)	Medium (commodity tooling)	Spreadsheet recalculation loop (§4.1)
Self-reflective	Model's structured re-evaluation	Moderate (no external oracle)	Low (protocol only)	Epistemic firewall (§4.2)
Planner-mediated	Typed intermediate artifacts checked against a plan	High on long chains	Medium (planner–executor split)	GAIA re-planning loop (§4.3)

Table 1: Three verification-oracle classes. The oracle sets the loop's reliability ceiling; the loop structure is shared.

3.4 The model mix: specialists inside the loop

Calling a frontier model at every loop stage is expensive and, for observe/compare, potentially counterproductive: the model that produced an artifact is primed to rationalize it [12, 7]. Leni therefore assigns each stage to the lightest model that performs it reliably, post-training small specialists for stages general models do poorly (Table 2). Specialists are 0.5–4B parameters, cheap enough to run on every iteration. All four are post-trained from open-weight Qwen3 bases (0.6B / 1.7B / 4B, Apache-2.0) with a shared recipe: distillation SFT from a Claude Opus 4.6 teacher (labels grounded in the strongest available oracle, rationales from the teacher), followed by task-specific reinforcement (RLVR/GRPO where a deterministic checker exists, for Cell-S, Parse-S, and Route-XS; SFT-then-DPO on preference pairs for Triage-S), 4-bit quantization for serving, and shadow deployment behind the live loop before promotion. Training data is Leni's production corpus: three years of enterprise back-office tasks (financial institutions and real estate operators)

and user accept/correct/reject judgments, collected independently of any benchmark campaign, plus synthetic augmentation derived from production scenario patterns. The authors attest that no benchmark items, and no corpora styled after the benchmarks, were used in training; the verification status of this attestation is discussed in §5.4. Weights and training data are proprietary; §10 lists this among the obstacles to independent replication.

Specialist	**Size**	**Post-training objective**	**Loop role**	**Used in**
Leni-Cell-S	∼4B	Spreadsheet cell-diff verification (recomputed value vs. intent)	Observe / Compare	Deterministic loop
Leni-Triage-S	∼4B	Premise decomposition + epistemic validity classification	Observe / Compare	Self-reflective loop
Leni-Parse-S	∼1.5B	Typed-artifact extraction from raw tool output	Observe (typing)	Planner-mediated loop
Leni-Route-XS	∼0.5B	Step-type classification for per-step routing	Dispatch / route	Planner-mediated loop

Table 2: The lightweight post-trained specialist mix. Because specialists run on every iteration, their cost and latency matter as much as their accuracy (§8).

4 Three Instantiations

All three instantiations run inside the same production system with no benchmark-specific modifications; they differ only in the oracle. One caveat before the details: while the *harness* is unmodified, individual components were built for production task families that overlap with what these benchmarks measure (a spreadsheet verifier is, unavoidably, aligned with a spreadsheet benchmark). The defensible claim is therefore not "untuned" but "not tuned *to these benchmarks*": the configurations are the ones serving production traffic, frozen before evaluation (§5.4).

4.1 Deterministic oracle: the recalculation loop

Spreadsheet editing is unforgiving: one wrong cell fails the task, and formula engines (Excel, LibreOffice, `openpyxl`) can evaluate the same formula differently, so a value that looks correct at write time can be silently wrong. The generating model is blind to this because `openpyxl` does not compute formulas.

The loop closes this gap: (a) a frontier executor (Claude Opus 4.6) edits and saves the workbook; (b) the server recalculates all formulas with LibreOffice headless; (c) values are read back through a separate deserialization path (`openpyxl`, `data_only=True`); (d) Leni-Cell-S compares recalculated non-empty cells against task intent and flags discrepancies; (e) on a flag, the frontier executor issues a targeted fix and Leni-Cell-S re-verifies. The loop runs at most two iterations, followed by a final recalculation pass so every formula cell carries a cached value for evaluation (Figure 2).

4.2 Self-reflective oracle: the epistemic firewall

Some tasks have no external oracle: no API returns "this methodology is fabricated." Here the loop uses structured re-evaluation as the oracle. Before answering, the agent must: (1) decompose the question into constituent claims; (2) classify each claim as known-valid, plausible-but-unrecognizable, or real-but-misapplied (both stages on Leni-Triage-S, trained to emit a hard accept/reject per claim rather than continue a fluent answer); (3) on a fabricated claim, produce a *constructive rejection* on

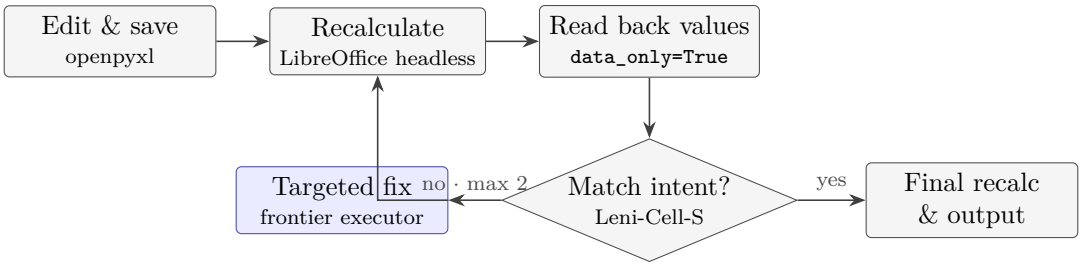

Figure 2: The deterministic recalculation loop. LibreOffice supplies external ground truth about formula evaluation; the small trained verifier (Leni-Cell-S, shaded) performs the comparison instead of the model that wrote the workbook.

the frontier model: name the fabrication, explain why, and redirect to the legitimate concept the user likely intended.

The mechanism (Figure 3) targets a specific observed failure: in raw single-pass failures the model often already encodes weak recognition of the fabrication (buried hedges, mid-answer caveats) but has no mechanism to promote that signal from a footnote to a behavioral rejection. The firewall supplies that mechanism.

Its principal risk is symmetric: a system trained to reject premises may over-reject legitimate but unusual ones. Measuring that false-positive rate requires a control set of valid-premise questions, which we have not yet run; until then, the firewall's *calibration* is unestablished even where its *sensitivity* is high (§10).

4.3 Planner-mediated oracle: re-planning over typed artifacts

Long-horizon tool tasks fail by cascade: a misread value on step two becomes a wrong filter on step three becomes a wrong answer on step six. Here the agent splits into a planner owning the global trajectory and an executor pool specialized per step type. Executors return *typed artifacts* (value, citation, confidence flag, error class) rather than raw text: Leni-Parse-S converts raw tool output into typed fields, and Leni-Route-XS selects the best model per step (fast/cheap for classification, frontier reasoning for multi-hop synthesis, strong grounding for vision) across Anthropic and OpenAI model families. The planner inspects each artifact against the original task and re-plans, inserting recovery steps or course corrections, before committing to the next step (Figure 4). In our GAIA run the planner re-planned on ∼38% of tasks, mostly to recover from failed sources.

5 Experimental Methodology

All evaluations use Leni's production configuration (the same harness serving user tasks across spreadsheets, documents, presentations, and analytics), with no benchmark-specific prompt or harness changes. We report exact counts alongside percentages, Wilson 95% confidence intervals for all proportions, and two-proportion z-tests where comparisons are made.

5.1 SpreadsheetBench Verified

Data: the 400-task expert-annotated Verified subset of SpreadsheetBench [1]. Tasks span formula computation, filtering, lookups, text processing, and multi-sheet operations. **Protocol:** pass@1, single attempt, no retries or cherry-picking; every cell in the designated answer region must exactly match the golden file (values read with `data_only=True`, compared after type coercion).

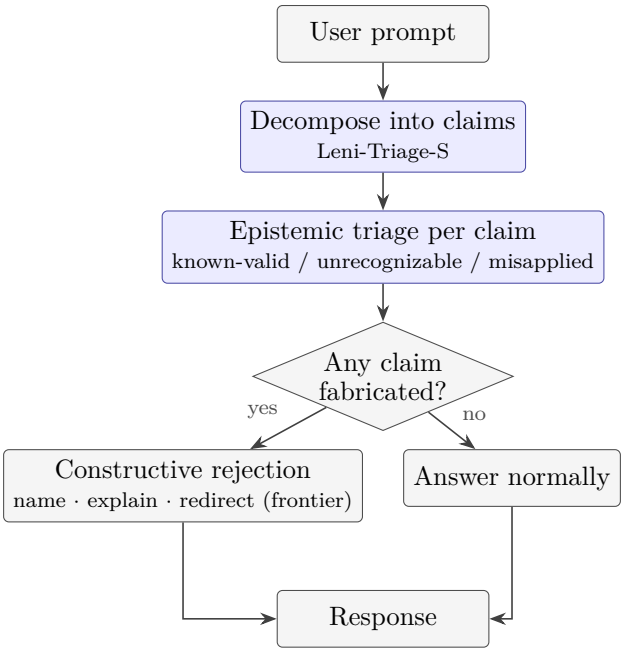

Figure 3: The self-reflective epistemic firewall. With no deterministic oracle available, structured re-evaluation of premises serves as the comparison stage; the shaded stages run on the small trained specialist.

Conditions: executor Claude Opus 4.6 (Anthropic API); read-back verification by Leni-Cell-S; sandboxed code execution; adaptive extended thinking; unmodified production spreadsheet system prompt. Evaluated 16–20 April 2026. **Baseline:** the 80.25% bare-model figure is the leaderboard's Claude Opus 4.6 entry (March 2026, minimal three-line prompt), not a rerun under our harness; the comparison is unpaired and treated accordingly.

5.2 BullshitBench v2

Data: 100 professionally framed questions containing fabricated premises across five domains (Software Engineering 40; Finance, Legal, Medical, Physics 15 each), using 13 deception techniques. **Protocol:** the benchmark's three-judge panel (GPT-4o, Gemini, and a Claude-class evaluator) scores each response 0–2; the panel score is the mean of the three judges, and a response counts as a correct rejection when the mean is at least 1.5. A 2 requires identifying the incoherence, making it central, and declining to answer within the false frame. **Conditions:** two configurations differing only in the frontier base model (Sonnet 4.6; Opus 4.6), triage on Leni-Triage-S in both. Evaluated 24 March 2026. **Caveats stated up front:** n=100; judge-based scoring with one intra-family judge; leaderboard entries (132 configurations at evaluation time) are raw API calls, so cross-entry rankings compare a system to models and are reported for context only, not as like-for-like.

5.3 GAIA validation

Data: GAIA validation split [2], 165 questions (Level 1: 53; Level 2: 86; Level 3: 26). **Protocol:** exact match on final answer, no partial credit. **Conditions:** planner on Claude Opus 4.6; executors routed across Claude Opus 4.6 / Sonnet 4.6 / Haiku 4.5 and OpenAI models by Leni-Route-XS; typed-artifact extraction by Leni-Parse-S; per-step adaptive extended thinking. **Caveats stated**

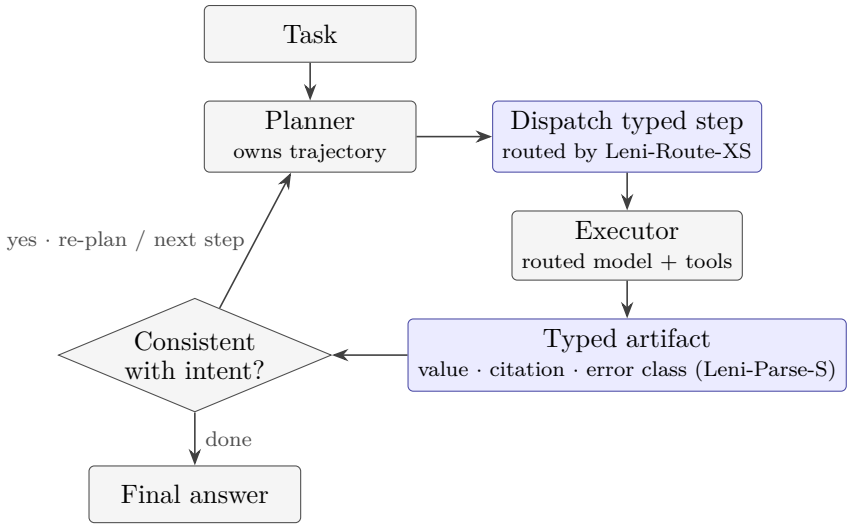

Figure 4: The planner-mediated loop. The typed-artifact interface makes re-planning tractable: the planner compares structured fields, not free text, before committing to the next step. Shaded stages run on small trained specialists.

up front: validation answers are public, so a web-searching agent could in principle retrieve them. We ran a scripted retrieval audit over all 803 stored trajectories (results and sensitivity bound in §6.3; script in the artifact bundle). The hidden test set has not yet been submitted to. Validation results here support the *decomposition*, not a leaderboard claim.

5.4 Contamination

Three contamination surfaces need separate treatment. First, base-model pretraining: we cannot rule it out for any publicly hosted benchmark; it affects the base and system arms symmetrically and is common to all published results. Second, GAIA validation answers are public and retrievable by a web-searching agent; we address this with a scripted trajectory audit and a conservative sensitivity bound (§6.3). Third, specialist post-training. The training data is Leni's production corpus, three years of enterprise back-office tasks and user accept/correct/reject judgments collected independently of any benchmark campaign, with synthetic augmentation derived from production scenario patterns. The authors attest that no benchmark items, and no corpora constructed to imitate the benchmarks, were used in any training set; production work encounters situations of the same broad kind (spreadsheet edits, premise-checking, tool chains) but not the benchmarks' tasks. We ran the audit's question-side scan (word 8/13-gram containment of every evaluation item, flag at $\geq$3 shared 8-grams or any 13-gram) over the production user-message corpus that sources the user-eval training signal: 30,104 messages against 365 evaluation items. Four messages flagged, matching three items (two GAIA validation questions, one DRACO problem) at 50–112 shared 8-grams, i.e. near-verbatim. All four trace to sessions from 2–17 March 2026 in which benchmark questions were manually entered during internal testing of the product, predating the recorded campaigns; three of the four originate from one internal account, and zero matches occur in organic customer traffic. Internal test accounts are excluded from training extraction (author attestation; the four flagged rows are published under pseudonymous internal-account labels, with underlying identifiers available to reviewers for verification), leaving zero evaluation items in the training-signal corpus. The scan script, thresholds, per-pair flags, and report are in the artifact repository; the

workbook-trace and tool-trajectory components of the corpus remain to be swept with the same script.

Evaluation practice. Reported results come only from full frozen runs. The released export preserves the complete run record, including development and superseded-configuration runs alongside the reported ones; configurations were not modified in response to results within a campaign, and no benchmark item influenced a prompt or a training set. The full run record, including unfavorable runs (a superseded L2 run at 62/86, two degraded L1 runs, and the pre-final SpreadsheetBench configurations), is preserved in the released export, which is the strongest evidence we can offer that results were not selected after the fact. The one reporting error we found, the mixed-selection 77.6% GAIA figure, arose in aggregation, not in running, and is corrected in §6.3.

6 Results

6.1 Headline: total system uplift, with uncertainty

Benchmark	Failure mode	Oracle	Base	Leni	Uplift
SpreadsheetBench (n=400)	silent computation	deterministic	80.25% [76.1, 83.9]	91.25% [88.1, 93.6]	+11.0 pp***
BullshitBench / Sonnet (n=100)	confabulation	self-reflective	91% [83.8, 95.2]	98% [93.0, 99.4]	+7 pp*
BullshitBench / Opus (n=100)	confabulation	self-reflective	87% [79.0, 92.2]	97% [91.5, 99.0]	+10 pp**
GAIA validation (n=165)	cascade error	planner-mediated	∼60%	75.2% [68.0, 81.1]	∼+15.2 pp

Table 3: Total system uplift over the bare base model. Brackets: Wilson 95% CIs. Significance from two-proportion z-tests ($^{*}p < 0.05$, $^{**}p < 0.01$, $^{***}p < 0.001$); the GAIA base figure is an internal estimate and is not significance-tested. These comparisons measure the whole architecture (tools, multiple passes, multiple models) against raw single-pass inference; §7 decomposes the total.

Table 3 summarizes. Two comparisons are statistically unambiguous: SpreadsheetBench (z=4.45, $p < 0.001$, unpaired) and BullshitBench on Opus. BullshitBench on Sonnet is nominally significant but rests on 7 discordant questions under judge-based scoring; we treat it as suggestive. The GAIA base-model figure (∼60%) comes from an internal single run of a minimal tool harness and should be read as an estimate.

We do not headline the claim that these uplifts "exceed the gap between frontier model releases": that comparison contrasts a system delta with a model delta on incommensurable configurations. The defensible statement is narrower and still useful: for a team holding the base model fixed, the architecture recovered more accuracy on these tasks than was available from any base-model swap we tested (Opus 4.6 vs. Sonnet 4.6 vs. GPT 5.4 as executors; internal runs).

6.2 SpreadsheetBench: the deterministic loop, fully instrumented

Leni scores 91.25% (365/400), which at evaluation time (April 2026 snapshot) placed second among public entries behind a neurosymbolic system with a custom spreadsheet runtime (DealGlass Tetra, 94.25%) and ahead of Nobie (91.00%), Qingqiu/Kingsoft (89.25%), Shortcut.ai (86.00%), bare Claude Opus 4.6 (80.25%), and GPT 5.4 strict (78.25%). Cell-level tasks pass at 92.4% (254/275), sheet-level at 88.8% (111/125); sheet-level operations (multi-region edits, cross-sheet references, structural changes) are harder, as expected.

The value of full instrumentation is the loop's confusion matrix (Table 4). The loop ran on 397 of 400 tasks (3 completed without triggering it; of those, 2 passed and 1 failed). Of the 397,

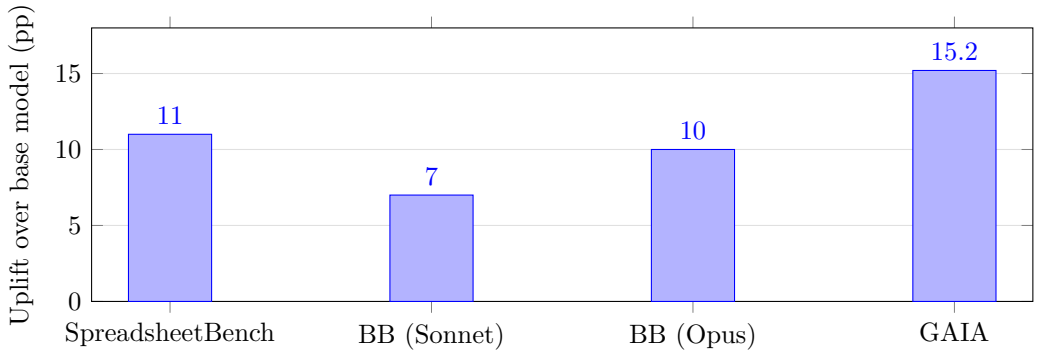

Figure 5: Total architectural uplift over the bare base model, by benchmark. The GAIA bar is measured against an internal base-model estimate (∼60%) and its pass@1 headline; see Table 3 for confidence intervals and §7 for the decomposition of these totals into structure, specialists, and verification.

the verifier confirmed 389 artifacts and flagged 8. All 8 flags were true errors (no observed false alarms); 6 were repaired and 2 were not. Of the 389 confirmations, 357 were correct and 32 were *false confirmations*: real errors the verifier missed.

	Artifact correct	**Artifact erroneous**
Verifier confirmed	357	32 (missed errors)
Verifier flagged	0 (no false alarms)	8 (6 repaired, 2 not)

Table 4: Task-level verifier confusion matrix on the 397 loop-triggering SpreadsheetBench tasks. Empirically: catch rate $c = 8/40 = 0.20$, fix rate $r = 6/8 = 0.75$, false-alarm rate $f = 0/357 = 0$ (bounded above by ∼1% at 95% confidence).

What the loop caught, and what it missed. The six rescued tasks share a signature: the recalculated values exposed defects that were invisible at write time, chiefly formulas that evaluate differently under LibreOffice than the executor assumed, cached-value mismatches, and format discrepancies that appear only after recalculation. Convergence under feedback looks stable rather than oscillatory: 26 tasks entered a second loop iteration after the agent modified its output on the first, and 22 of the 26 passed. A full run of the same configuration two days earlier (14 April) was degraded by API-server downtime and scored 356/400; the completed re-run is the reported result. The two runs disagreed on 11 tasks: 10 fail→pass, consistent with recovery from the infrastructure failures, and 1 pass→fail, consistent with sampling variance. This bounds run-to-run sensitivity, infrastructure faults included, at about ±3 pp. The 32 missed errors have not yet been individually coded by failure locus (comparator vs. recalculation-oracle vs. task interpretation); that coding is the immediate next step in the loop's telemetry, because it determines whether the remaining path to the neurosymbolic leader runs through a better comparator or a better executor.

Three observations. First, in the notation of §3.2, the empirical operating point is $c \approx 0.20$, $r \approx 0.75$, $f \approx 0$: the verifier is conservative; it flags rarely, but its flags are precise and its fixes usually land. With $f \approx 0$, Eq. (1) guarantees the loop cannot hurt, and the measured rescue (+6 tasks net, +1.5 pp) matches $(1 - p)\, c\, r$ within rounding. Second, the 32 missed errors quantify the cost of LLM-mediated comparison relative to symbolic enforcement: a symbolic comparator would have caught a large fraction of them, and this is a plausible mechanism for most of the 3-point gap

to the neurosymbolic leader. Third, the ceiling on further gains from this loop is visible in its own telemetry: raising c (a better-trained or symbolically assisted comparator) is worth up to +8 pp; raising r is worth at most +0.5 pp. Instrumenting the loop tells you where to invest.

6.3 GAIA: planner-mediated loop, and what the tier data can support

A correction, first. An earlier company report gave 77.6% (128/165) on this split. In preparing this paper we exported every stored GAIA trajectory from the production evaluation database (803 validation runs across seven campaigns, March–April 2026) and re-graded all of them with the official GAIA scorer; the re-grader agrees with all 218 runs that carried grades at export time (218/218). The re-grading shows the 77.6% figure mixed selection rules across tiers: the Level 1 component (47/53) was best-of-three over repeated runs, and the Level 3 component drew on more than one campaign, while Level 2 (65/86) was a legitimate single attempt. We therefore report both quantities cleanly and retire the earlier figure.

Under a pre-specified pass@1 rule (the last completed attempt per task within the final-configuration campaigns), Leni scores **75.2% (124/165)** [68.0, 81.1]: Level 1, 44/53 [70.8, 90.8]; Level 2, 65/86 [65.5, 83.4]; Level 3, 15/26 [38.9, 74.5]. Best-of-all-runs across campaigns reaches 83.0% (137/165), a pass@k quantity with k varying by tier that we report for completeness, not comparison. Because Level 1 was run in full four times, GAIA also yields direct run-to-run variance: complete runs scored 44, 43, 43, and 43 of 53, with five tasks flipping outcome between the two closest runs, and two additional runs degraded by infrastructure failures (3–4 unanswered tasks each) that we report rather than exclude.

Retrieval audit. Because validation answers are public, we scanned every stored trajectory for URLs of known GAIA-derived sources (the HF dataset pages, the GAIA paper, WebVoyager's `GAIA_web.jsonl`, agents-course `metadata.jsonl`). Twenty-five of 803 trajectories, spanning 12 tasks, contain such a URL; in nearly all cases it appears inside a web-search result list rather than as a fetched page, but URL presence cannot by itself exclude use. Seven runs in the pass@1 selection are flagged, all seven scored correct. Treating every flagged correct run as a failure gives a conservative contamination-adjusted lower bound of $117/165 = 70.9\%$. The audit script and per-run flags ship with the artifact bundle so the boundary between "URL seen" and "answer used" can be adjudicated by anyone from the raw trajectories.

For context, contemporaneous public figures reconstructed from per-level reports place Genspark near 75.4%, Manus near 73.4%, and OpenAI Deep Research near 67.4% on this split. At 75.2% pass@1, Leni is statistically indistinguishable from Genspark and Manus, and nominally but not significantly ahead of Deep Research ($z \approx 1.6$, $p \approx 0.12$, against a reconstructed figure). We make no superiority claim on this benchmark. The hidden test set (n=300, evaluation server) has not been submitted to; it remains the decisive surface for any cross-system claim, and we commit to reporting the result when the submission completes, whether or not it flatters the system.

What the GAIA data does establish is internal: the architecture's uplift over the ∼60% bare-tool-use estimate is roughly +15 pp at pass@1, and the per-level telemetry (planner re-plans on ∼38% of tasks, mostly source-failure recovery) feeds the decomposition in §7.

6.4 BullshitBench: sensitivity established, calibration not yet

Leni reaches 98% correct rejection on Sonnet 4.6 (mean judge score 1.963/2.0, one failure) and 97% on Opus 4.6 (1.950/2.0, two failures), against 91% for the best raw model configuration (Claude Sonnet 4.6 High, mean 1.87) and near-coin-flip performance for the best OpenAI and

Google configurations (GPT 5.4: 48%; Gemini 3 Pro Preview: 48%). Domain-level rates are uniform (93–100% across Finance, Legal, Medical, Physics, Software Engineering), consistent with a behavioral disposition induced by the firewall rather than per-domain knowledge.

Valid-premise control. A firewall trained to reject premises could inflate rejection scores by over-rejecting, so sensitivity alone is not enough. We measure the other side of the boundary with DRACO, an internal rubric-graded corpus of 100 legitimate, professionally difficult, jargon-dense questions (finance, law, medicine, econometrics, technology, among others; real frameworks such as Goodman-Bacon decomposition and Callaway–Sant'Anna estimators) evaluated through the same firewall-active production harness on 16–17 April 2026, two runs per task, 201 scored runs. The system engaged substantively with every question: mean rubric score 71.3% (median 73.5%), no run below 10% and no task below 20% at task level, where a premise-rejection response would score near zero on the factual-accuracy rubrics. Zero over-rejections in 100 tasks bounds the firewall's false-positive rate on this distribution at roughly 3.6% (95% upper limit, rule of three). The remaining gap is distributional: DRACO questions are hard and jargon-heavy but were not constructed adversarially to mimic the benchmark's 13 deception techniques with valid premises; that matched-style control is the outstanding experiment, and until it runs, calibration against *deliberately fabrication-like* valid premises remains open.

Two further qualifications. First, the +7 pp Sonnet-configuration uplift is 7 questions under judge scoring with one intra-family judge, while the Opus-configuration uplift (+10 pp) rests on firmer ground. And "above all 132 public configurations" compares an agentic system against raw API calls; we report it as context for the size of the architectural effect, not as a like-for-like ranking.

Where the two configurations diverge. Bucket assignments differed on three of the 100 items, and the divergences are informative about what the firewall does not control: on phys_st_01 (Physics, metrology) Opus validated a fabricated instrument class while Sonnet rejected it; on fin_nn_02 (Finance, CECL) Opus hedged without committing while Sonnet cleanly separated valid from invalid methods; on leg_tce_01 (Legal, contracts) Sonnet returned an empty response while Opus produced a substantive rejection. The firewall raises both configurations to near-ceiling but does not erase base-model-specific vulnerabilities; per-task judge scores for all 100 items are retained and available.

The mechanistically interesting observation survives all three qualifications: extended-thinking configurations of strong reasoners perform *worse* than their standard counterparts on this benchmark, consistent with inference-time compute being spent constructing a path to an answer inside a false frame rather than questioning the frame. The firewall redirects compute toward an explicit checkpoint. Reliability comes from where the model is forced to look, not only from how much it thinks.

7 Decomposing the Uplift

Table 5 decomposes total uplift by architectural layer. SpreadsheetBench is fully decomposed under the frozen harness; GAIA layer figures are from internal ablation runs (complete controlled ablation forthcoming); BullshitBench is not layer-decomposed because the firewall *is* the scaffold.

What the decomposition does and does not show. The verification loop is not the majority contributor; structure is. But the loop's contribution is positionally concentrated: on Spreadsheet-Bench, +1.5 pp is the difference between a mid-pack and a near-top result, because every system at the top has already exhausted the gains from good scaffolding. We previously believed the GAIA

Benchmark	Base	+ Structure	+ Verification loop	Loop-isolated
SpreadsheetBench	80.25%	89.75% (prompt + scaffold)	91.25%	+1.5 pp (6 rescues)
GAIA	∼60%	∼70% (+ planner–executor) → ∼74% (+ routing)	75.2%	∼+1 pp (vs. estimated tier)
BullshitBench (Opus)	87%	—	97%	+10 pp (whole firewall)

Table 5: Uplift by layer. Most of the total uplift comes from structure (prompting, planning, routing); the verification step's isolated contribution is small. The GAIA structure tiers are internal estimates whose selection rules were not recorded, so the GAIA loop-isolated figure is indicative only.

data showed the loop's contribution growing with chain length, as Eq. (1) predicts; after correcting the GAIA score to pass@1, the loop's GAIA increment (∼+1 pp against an estimated structure tier) is no longer distinguishable from its spreadsheet increment. The chain-length prediction stands untested, and the controlled GAIA ablation needed to test it is the natural companion experiment to the swap ablation below.

Who observes matters: the specialist-swap ablation. Holding the loop structure fixed and swapping the observe/compare stage from the trained specialist back onto the frontier model that generated the artifact: on SpreadsheetBench the rescue count falls from 6 tasks to 2 (the generator, asked to verify cells it just wrote, tends to rationalize them rather than flag them), and on BullshitBench correct rejection falls by 4–5 pp (the specialist emits a hard accept/reject per claim where the general model continues a fluent answer). Two boundaries on this evidence: the swaps are single runs from the internal engineering evaluation of the specialist mix, covering Cell-S and Triage-S only (the Parse-S and Route-XS swaps have not been run), and the design lacks a third verifier condition, an independent frontier model from a different provider that did not generate the artifact, which is the cell that would separate *independence* from *specialization*. Within those limits, the results fit documented self-assessment biases [12, 7] and support what we consider the paper's most consequential claim: *the loop's value comes from the independence and specialization of the observer*.

7.1 Internal configuration tiers, and the baselines still missing

Comparing the full system to a bare base model measures product uplift, not mechanism. Two internal tiers function as partial scaffold baselines because they hold the model, tools, and sandbox fixed: on SpreadsheetBench, the prompt-plus-self-validation configuration (89.75%) is a same-model scaffold with no external oracle, and on GAIA the planner–executor tier (∼70%) and routing tier (∼74%) isolate structure from verification. What we have not run are published external scaffolds, ReAct-style [9], CRITIC-style [8], and best-of-n at matched token budget, under identical tool access. Without them, the bare-model comparison bounds the architecture's value from above rather than pinning down its advantage over cheaper alternatives; these runs are scoped and listed in §10.

8 Cost and Latency

Per-step verification is only viable if the verifier is cheap. Per call, the specialists serve at a small fraction of the cost of the same operation on a frontier model (internal serving estimates: ∼0.1× for Leni-Cell-S and Leni-Triage-S, ∼0.05× for Leni-Parse-S, ∼0.02× for Leni-Route-XS, all quantized to 4 bits), which is what makes it economical to run the compare stage on all 397 loop-triggering

SpreadsheetBench tasks. End-to-end latency overhead of the loop has not been measured for publication and belongs in the artifact release. On GAIA, routing by Leni-Route-XS *reduces* net cost: cheap steps route to small models, funding extended reasoning on hard steps; internal estimates attribute ∼4 pp of GAIA accuracy to routing at net-negative cost. Small trained specialists are what make "verify every step" an affordable default rather than a luxury.

9 Discussion

Reliability is mostly an architecture problem. For a fixed base model, the architecture recovered +7 to ∼+15 pp across three unrelated failure modes, and the decomposition attributes this to three separable mechanisms: scaffolding/structure (largest share), specialist staffing (necessary for the loop's value; cheap), and the verification checkpoint itself (small, positionally decisive). Teams should invest in that order, but not skip the third: at the top of a leaderboard, or at the tail of a reliability SLA, the checkpoint is where the remaining failures are.

The oracle sets the ceiling; the observer sets the floor. A commodity deterministic oracle (LibreOffice recalculation) leaves only a 3-point gap to a full neurosymbolic system at a fraction of the engineering cost, and our confusion matrix localizes that gap in the LLM-mediated comparison stage (32 missed errors). Where re-execution is possible, prefer it, and consider symbolically assisted comparison. Where it is not, an independent specialized observer still outperforms self-assessment by the generator; the swap ablation and the prior literature agree on this point.

Why more thinking is not enough. Extended-thinking reasoners can do *worse* on premise validation: compute is spent building a path to an answer inside a false frame. A checkpoint that forces the model to look at the frame beats additional compute spent inside it. This is the practical content of Huang et al.'s negative result [7] for enterprise deployment: self-correction needs an external or structurally independent signal, and the architecture's job is to supply one.

10 Limitations and Threats to Validity

- **Vendor evaluation.** This is an evaluation of Leni by Leni's builders. We mitigate with public benchmarks, exact-match protocols where available, frozen configurations, full counts, and stated CIs, but independent replication is the real remedy; we will provide evaluation transcripts to qualified researchers on request.
- **Limited repeated-run coverage.** The headline SpreadsheetBench and BullshitBench results are single scored runs. Measured run-to-run sensitivity where repeats exist: GAIA Level 1 complete runs scored 44/43/43/43 of 53 (5 tasks flipped between adjacent runs), and a SpreadsheetBench run interrupted by API downtime and its completed re-run disagreed on 11 of 400 tasks (10 attributable to the infrastructure failures). Differences smaller than ∼3 pp should be treated as unresolved.
- **GAIA validation, not test; corrected headline.** Validation answers are public and the hidden test set has not been submitted to. The scripted retrieval audit (§6.3) brackets the pass@1 result at [70.9%, 75.2%] depending on how flagged trajectories are adjudicated. The previously reported 77.6% mixed best-of-k and pass@1 selection and is superseded. Cross-system GAIA statements in this paper are context, not claims.

- **Training exclusion audited on the question corpus; two components remain.** The published n-gram sweep of the production message corpus found 4/30,104 matches, all adjudicated as internal manual benchmark testing with identifiers published for verification (§5.4). Remaining on attestation: that internal test accounts are excluded from training extraction, and the sweeps of the workbook-trace and tool-trajectory corpus components.
- **No external scaffold baselines.** ReAct-style, CRITIC-style, and best-of-n configurations at matched tool access and token budget have not been run (§7.1); the bare-model baseline overstates the architecture's advantage over cheaper scaffolds by an unknown amount.
- **Competitor figures are reconstructions.** GAIA competitor overall scores are weighted estimates from per-level reports at a point in time; rankings shift. All rank statements in this paper are dated snapshots (April 2026), not standing claims.
- **Valid-premise control is observational, not adversarially matched.** The DRACO control (§6.4) bounds over-rejection at $\lesssim$3.6% on legitimate expert-level questions, but its items were not styled after the benchmark's deception techniques; a matched-style control with deliberately fabrication-like valid premises remains to be run.
- **Judge-based scoring.** The BullshitBench panel spans three providers but includes one judge from the same model family as the systems under test; intra-family preference is documented [12, 11].
- **Preliminary ablations.** The specialist-swap results cover two of four specialists, come from internal single runs, and lack the independent-generalist verifier condition; the GAIA layer decomposition is likewise from internal single runs. They motivate, but do not yet establish, the "independent observer" hypothesis.
- **Proprietary components.** Specialist weights, the production training corpus, and the production prompts are not released. The base models, sizes, and full post-training recipe are documented (§3.4), which permits method-level replication on public data, but not verification of our checkpoints; we flag this as the main obstacle to independent verification of the mechanism as opposed to the scores.

11 Conclusion

Across three benchmarks stressing three unrelated failure modes, a production agent architecture (verification loops staffed by lightweight post-trained specialists, on top of strong scaffolding) produces large total uplift over its frontier base model, with the two largest comparisons statistically unambiguous. Decomposition, not the headline numbers, is the durable contribution: structure supplies most of the gain; the verification checkpoint supplies a small, positionally decisive remainder; and preliminary evidence suggests the checkpoint only pays when the observer is independent of the generator. The verifier confusion matrix we report turns loop design from folklore into measurement: catch rate, fix rate, and false-alarm rate are estimable quantities that tell a team exactly where the next point of reliability will come from.

For teams building enterprise agents where a confidently wrong answer has real cost, the actionable guidance is concrete and ranked: invest first in structure (planning, routing, typed interfaces), then staff observation with a small model that did not generate the artifact, then give the loop the strongest oracle the task admits: re-execution where possible, structured re-evaluation where not. And instrument the loop: its own telemetry will tell you whether to believe it.

Artifact Availability and Run Configuration

All results in this paper were computed from a CSV export of the production evaluation database: per-run records for SpreadsheetBench (1,299 runs over 400 tasks across four configurations, with per-task pass/fail and instruction type), BullshitBench (500 runs over 100 items, of which 200 carry per-judge scores, panel means, and bucket assignments), GAIA (803 validation runs over 165 tasks across seven campaigns, with stored final answers per trajectory), and the DRACO valid-premise control (309 runs over 100 tasks with per-criterion rubric evaluations). The GAIA re-grading script implements the official scorer and reproduces all 218 database-resident grades exactly; it ships with the export. The complete bundle (run records, audit scripts, and reports, including unfavorable and superseded runs) is publicly released at `https://github.com/arnabdastidar/leni-agent-evals`; the loop-action telemetry of Table 4 currently exists in the run transcripts rather than as structured fields and is being extracted for the release. Specialist weights and the production training corpus are proprietary, but the method is documented to replication depth: Qwen3 bases (0.6B/1.7B/4B, Apache-2.0), distillation SFT from a Claude Opus 4.6 teacher, RLVR/GRPO against deterministic checkers (Cell-S, Parse-S, Route-XS) or SFT-then-DPO on preference pairs (Triage-S), QLoRA iteration with full-fine-tune release checkpoints, 4-bit serving, and shadow deployment before in-loop promotion.

Run configuration: SpreadsheetBench used Claude Opus 4.6 via the Anthropic API with sandboxed code execution and adaptive extended thinking, the unmodified production xlsx prompt, evaluated 16–20 April 2026. BullshitBench used the production harness on Claude Sonnet 4.6 and Claude Opus 4.6, evaluated 24 March 2026 under the benchmark's three-judge panel and bucket rule. GAIA used a Claude Opus 4.6 planner with executors routed across Claude Opus 4.6, Sonnet 4.6, Haiku 4.5, and OpenAI models, with per-step adaptive extended thinking. Sampling parameters and harness commit identifiers are not reported here and will accompany the run-log release.

Conflict of Interest and Authorship

The corresponding author is the chief executive and founder of Leni Inc., which develops the system evaluated here; the Leni team built the production system, ran the evaluation campaigns, and trained the specialist models. All claims, analyses, and errors are the responsibility of the corresponding author. This work has not been independently replicated; the evaluation counts, per-task outcomes, and audit scripts are publicly released at `https://github.com/arnabdastidar/leni-agent-evals`.

References

[1] Ma, Z., Zhang, B., Zhang, J., Yu, J., Zhang, X., Zhang, X., Luo, S., Wang, X., Tang, J. (2024). SpreadsheetBench: Towards Challenging Real World Spreadsheet Manipulation. *NeurIPS 2024 Datasets and Benchmarks Track* (Spotlight).

[2] Mialon, G., Fourrier, C., Swift, C., Wolf, T., LeCun, Y., Scialom, T. (2023). GAIA: A Benchmark for General AI Assistants. *arXiv:2311.12983*.

[3] Gostev, P. (2026). Bullshit Benchmark (v2). `https://github.com/petergpt/bullshit-benchmark`.

[4] Madaan, A., et al. (2023). Self-Refine: Iterative Refinement with Self-Feedback. *NeurIPS 2023*.

[5] Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., Yao, S. (2023). Reflexion: Language Agents with Verbal Reinforcement Learning. *NeurIPS 2023*.

[6] Wang, X., et al. (2023). Self-Consistency Improves Chain of Thought Reasoning in Language Models. *ICLR 2023*.

[7] Huang, J., et al. (2024). Large Language Models Cannot Self-Correct Reasoning Yet. *ICLR 2024*.

[8] Gou, Z., Shao, Z., Gong, Y., Shen, Y., Yang, Y., Duan, N., Chen, W. (2024). CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing. *ICLR 2024*.

[9] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., Cao, Y. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. *ICLR 2023*.

[10] Ridnik, T., Kredo, D., Friedman, I. (2024). Code Generation with AlphaCodium: From Prompt Engineering to Flow Engineering. *arXiv:2401.08500*.

[11] Zheng, L., et al. (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *NeurIPS 2023 Datasets and Benchmarks Track*.

[12] Panickssery, A., Bowman, S. R., Feng, S. (2024). LLM Evaluators Recognize and Favor Their Own Generations. *NeurIPS 2024*.